



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“Semana 4: Listas Dobles y Circulares”

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

Fase 1:COMPRENDE.....	3
1 Visualización de Lista Doblemente Enlazada.....	3
Checkpoint Metacognitivo - Fase COMPRENDE.....	4
Fase 2: APLICA.....	4
2 Proyecto: Playlist Bidireccional con Lista Doble propia.....	4
Checkpoint Metacognitivo - Fase APLICA.....	11
Fase 3:REFLEXIONA.....	12
Checkpoint Metacognitivo - Fase REFLEXIONA.....	12
Fase 4: VALIDA.....	12
3 El Auditor de Listas Dobles.....	12
Checkpoint Metacognitivo - Fase VALIDA.....	16
FASE 5: PROFUNDIZA.....	16
Checkpoint Metacognitivo - Profundiza.....	16

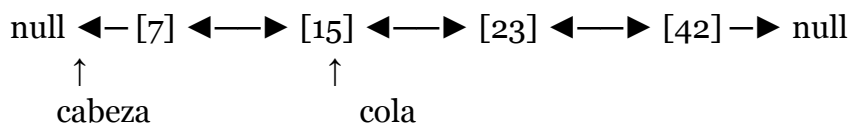
Fase 1: COMPRENDE

1 Visualización de Lista Doblemente Enlazada

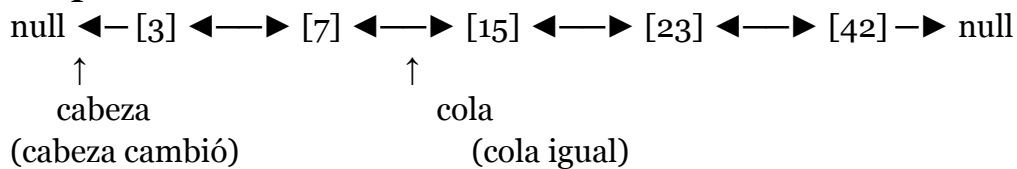
Documento con los 3 diagramas anotados por ti (puedes hacerlos en papel, ASCII o herramienta digital). Incluye una explicación con tus propias palabras de por qué eliminar un nodo conocido es $O(1)$ en la lista doble pero $O(n)$ en la lista simple.

1) Insertar 3 al inicio

Antes

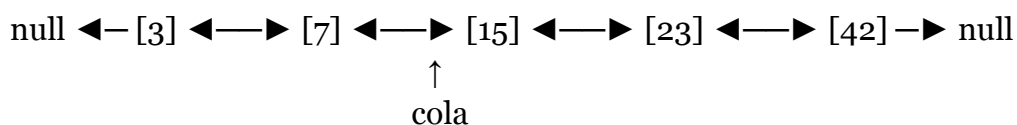


Después

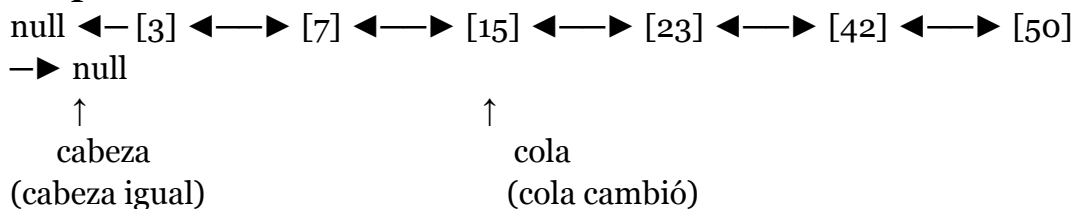


2) Insertar 50 al final

Antes

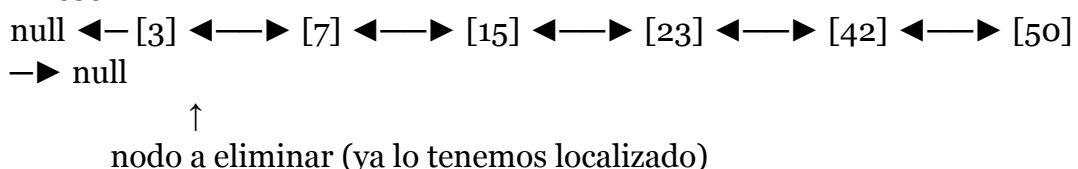


Después

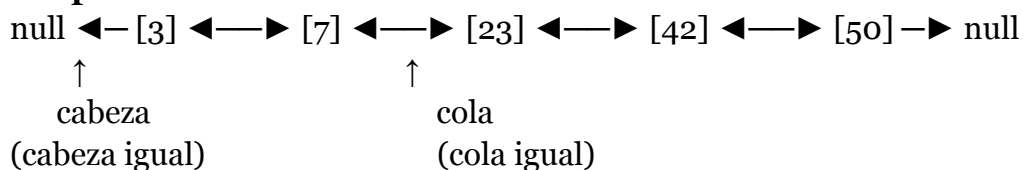


3) Eliminar nodo de valor 15

Antes



Después



¿Por qué eliminar un nodo conocido es $O(1)$ en la lista doble pero $O(n)$ en la lista simple?

Es $O(1)$ por ser un número fijo de operaciones, mientras que en la lista simple como no sabes quien es el anterior debes recorrer toda la lista nuevamente hasta encontrar el nudo.

Checkpoint Metacognitivo - Fase COMPRENDE

¿Puedo dibujar de memoria la estructura de una lista doble con 3 nodos, incluyendo cabeza, cola, y todos los punteros?

Si, es bastantes simple al ya haber trabajado con varios ejemplos

¿Puedo explicar con mis palabras por qué eliminarFinal() es $O(1)$ en lista doble pero $O(n)$ en lista simple?

es $O(1)$ por que puedes regresar en la lista hacia adelante y hacia atras, mientras que es $O(n)$ por que te obliga a recorrer todo el nodo

¿Entiendo los 5 invariantes listados arriba? ¿Puedo dar un ejemplo de qué pasa si alguno se rompe?

Aún no, hay invariantes que aun me faltan por contemplar.

Fase 2: APLICA

2 Proyecto: Playlist Bidireccional con Lista Doble propia

Código fuente del sistema Playlist Bidireccional con la lista doble construida por ti.

Incluye:

(1) la clase NodoDoble y la estructura ListaDoble con mínimo 10 métodos funcionales

(2) la clase Playlist con los 5 métodos del escenario

(3) una demostración que ejecute una secuencia de operaciones mostrando inserción, navegación adelante/atrás, eliminación de canción actual y recorrido completo

(4) comentarios explicando la lógica de cada bloque y cómo el puntero "actual" se mantiene consistente.

```
#include <iostream>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

// Nodo de una lista doble: tiene un valor y apunta al anterior y al siguiente.

```
class NodoDoble {
```

```
public:
```

```
    string valor;
```

```
    NodoDoble* prev;
```

```
    NodoDoble* next;
```

```
    // Al crear un nodo, no está conectado a nada.
```

```
    explicit NodoDoble(const string& v)
```

```
        : valor(v), prev(nullptr), next(nullptr) {}
```

```
};
```

```
class ListaDoble {
```

```
private:
```

```
    // Evita copias (para no borrar memoria dos veces).
```

```
    ListaDoble(const ListaDoble&) = delete;
```

```
    ListaDoble& operator=(const ListaDoble&) = delete;
```

```
public:
```

```
    NodoDoble* cabeza;
```

```
    NodoDoble* cola;
```

```
    int tamaño;
```

```
    // Lista vacía al inicio.
```

```
    ListaDoble() : cabeza(nullptr), cola(nullptr), tamaño(0) {}
```

```
    // Libera todos los nodos al final del programa (evita fugas de memoria).
```

```
    ~ListaDoble() {
```

```
        NodoDoble* actual = cabeza;
```

```
        while (actual != nullptr) {
```

```
            NodoDoble* sig = actual->next;
```

```
            delete actual;
```

```
            actual = sig;
```

```
        }
```

```
        cabeza = nullptr;
```

```
        cola = nullptr;
```

```
        tamaño = 0;
```

```
    }
```

```
    bool estaVacía() const { return cabeza == nullptr; }
```

```
    int size() const { return tamaño; }
```

```
    NodoDoble* getCabeza() const { return cabeza; }
```

```
    NodoDoble* getCola() const { return cola; }
```

```

// Inserta un nuevo nodo al inicio.
void insertarInicio(const string& valor) {
    NodoDoble* nuevo = new NodoDoble(valor);

    if (cabeza == nullptr) {
        // Si estaba vacía, nuevo es cabeza y cola.
        cabeza = nuevo;
        cola = nuevo;
    } else {
        // Conecta: nuevo <-> vieja cabeza.
        nuevo->next = cabeza;
        cabeza->prev = nuevo;
        cabeza = nuevo;
    }

    tamaño++;
}

```

```

// Inserta un nuevo nodo al final.
void insertarFinal(const string& valor) {
    NodoDoble* nuevo = new NodoDoble(valor);

    if (cabeza == nullptr) {
        // Si estaba vacía, nuevo es cabeza y cola.
        cabeza = nuevo;
        cola = nuevo;
    } else {
        // Conecta: vieja cola <-> nuevo.
        nuevo->prev = cola;
        cola->next = nuevo;
        cola = nuevo;
    }

    tamaño++;
}

```

```

// Elimina el primer nodo.
void eliminarInicio() {
    if (cabeza == nullptr) return; // no hay nada

    if (cabeza == cola) {
        // Solo hay uno.
        delete cabeza;
    }
}

```

```

    cabeza = nullptr;
    cola = nullptr;
    tamaño--;
    return;
}

// Hay 2 o más.
NodoDoble* viejo = cabeza;
cabeza = viejo->next;
cabeza->prev = nullptr; // la nueva cabeza no tiene anterior
delete viejo;
tamaño--;
}

// Elimina el último nodo.
void eliminarFinal() {
    if (cabeza == nullptr) return;

    if (cabeza == cola) {
        // Solo hay uno.
        delete cola;
        cabeza = nullptr;
        cola = nullptr;
        tamaño--;
        return;
    }

    // Hay 2 o más.
    NodoDoble* viejo = cola;
    cola = viejo->prev;
    cola->next = nullptr; // la nueva cola no tiene siguiente
    delete viejo;
    tamaño--;
}

// Elimina el nodo indicado (ya tenemos el puntero).
void eliminarNodo(NodoDoble* nodo) {
    if (nodo == nullptr) return;

    if (nodo == cabeza) { eliminarInicio(); return; }
    if (nodo == cola) { eliminarFinal(); return; }

    // Está en medio: une a sus vecinos y lo borra.
    NodoDoble* a = nodo->prev;

```

```

    NodoDoble* b = nodo->next;
    a->next = b;
    b->prev = a;

    delete nodo;
    tamaño--;
}

// Busca por nombre y devuelve el primer nodo encontrado (o nullptr).
NodoDoble* buscar(const string& valor) const {
    NodoDoble* actual = cabeza;
    while (actual != nullptr) {
        if (actual->valor == valor) return actual;
        actual = actual->next;
    }
    return nullptr;
}

// Devuelve el contenido de la lista de izquierda a derecha.
vector<string> recorrerAdelante() const {
    vector<string> resultado;
    NodoDoble* actual = cabeza;
    while (actual != nullptr) {
        resultado.push_back(actual->valor);
        actual = actual->next;
    }
    return resultado;
}

// Devuelve el contenido de la lista de derecha a izquierda.
vector<string> recorrerAtras() const {
    vector<string> resultado;
    NodoDoble* actual = cola;
    while (actual != nullptr) {
        resultado.push_back(actual->valor);
        actual = actual->prev;
    }
    return resultado;
}
};

class Playlist {
private:
    // La playlist usa internamente una lista doble.

```


ListaDoble lista;

public:

// "actual" apunta a la canción que se está reproduciendo.

// Si no hay canciones, actual es nullptr.

NodoDoble* actual;

Playlist() : lista(), actual(nullptr) {}

// Agrega una canción al final.

// Si es la primera canción, también se vuelve la actual.

void agregarCancion(const string& nombre) {

 lista.insertarFinal(nombre);

 if (actual == nullptr) actual = lista.getCabeza();

}

// Avanza a la siguiente canción.

string siguiente() {

 if (actual == nullptr) return "playlist vacía";

 if (actual->next == nullptr) return "fin de playlist";

 actual = actual->next;

 return actual->valor;

}

// Regresa a la canción anterior.

string anterior() {

 if (actual == nullptr) return "playlist vacía";

 if (actual->prev == nullptr) return "inicio de playlist";

 actual = actual->prev;

 return actual->valor;

}

// Elimina la canción actual.

// Regla: intenta moverse a la siguiente; si no existe, se va a la anterior.

string eliminarActual() {

 if (actual == nullptr) return "playlist vacía";

 // Guardamos a dónde nos moveremos ANTES de borrar.

 NodoDoble* candidato = actual->next;

 if (candidato == nullptr) candidato = actual->prev;

 lista.eliminarNodo(actual); // aquí se hace delete del nodo

 actual = candidato;

```

        if (actual == nullptr) return "playlist vacía";
        return actual->valor;
    }

    // Imprime la playlist y marca la actual con ">>".
    void mostrarPlaylist() {
        NodoDoble* it = lista.getCabeza();

        if (it == nullptr) {
            cout << "[vacía]\n";
            return;
        }

        while (it != nullptr) {
            cout << (it == actual ? ">> " : " ") << it->valor << "\n";
            it = it->next;
        }
    }

    // Solo para la demo: muestra recorridos completos.
    void mostrarRecorridos() const {
        auto a = lista.recorrerAdelante();
        auto r = lista.recorrerAtras();

        cout << "Adelante: ";
        for (const auto& s : a) cout << s << " ";
        cout << "\n";

        cout << "Atrás: ";
        for (const auto& s : r) cout << s << " ";
        cout << "\n";
    }
};

int main() {
    Playlist p;

    cout << "== Inicial ==\n";
    p.mostrarPlaylist();

    cout << "\n== Insertar A, B, C ==\n";
    p.agregarCancion("A");
    p.agregarCancion("B");
    p.agregarCancion("C");
}

```

```

p.mostrarPlaylist();

cout << "\n== Siguiente ==\n";
cout << "siguiente(): " << p.siguiente() << "\n";
p.mostrarPlaylist();

cout << "siguiente(): " << p.siguiente() << "\n";
p.mostrarPlaylist();

cout << "siguiente(): " << p.siguiente() << "\n"; // fin
p.mostrarPlaylist();

cout << "\n== Anterior ==\n";
cout << "anterior(): " << p.anterior() << "\n";
p.mostrarPlaylist();

cout << "\n== Eliminar actual ==\n";
cout << "eliminarActual(): " << p.eliminarActual() << "\n";
p.mostrarPlaylist();

cout << "\n== Recorridos completos ==\n";
p.mostrarRecorridos();

cout << "\n== Vaciar eliminando actual ==\n";
cout << "eliminarActual(): " << p.eliminarActual() << "\n";
p.mostrarPlaylist();
cout << "eliminarActual(): " << p.eliminarActual() << "\n";
p.mostrarPlaylist();

return 0;
}

```

Checkpoint Metacognitivo - Fase APLICA

¿Puedo implementar eliminarNodo() sin mirar el pseudocódigo? ¿Qué parte me cuesta más?

No, aun tengo muchos problemas con el código, aunque el pseudocódigo halla ayudado, tuve que recurrir a la IA para que me diera ciertas respuestas como vector y donde acomodar cada parte del código de ser necesario.

¿Cuántos punteros debo actualizar al insertar en una lista doble vs. una simple? ¿Puedo listarlos?

pueden ser 2 y 2, con la lista enlazada son insertar al inicio y al final, con la lista doblemente enlazada es igual, insertar al inicio, insertar al final.

Si la IA me dio código, ¿lo entiendo línea por línea o hay partes que "confío" sin verificar? Como lo decía, me dio la ia la biblioteca de vector, por eso la investigue, como también me ayudó a estructurar ciertas partes del codigo que si puedo leer

Fase 3: REFLEXIONA

Checkpoint Metacognitivo - Fase REFLEXIONA

¿Puedo explicar a un compañero cuándo elegir lista doble vs. simple vs. arreglo, con ejemplos concretos?

Escoger el arreglo cuando se necesite acceso por índice rápido, lista simple solo cuando se necesite recorrer hacia adelante y listas doble cuando se necesite recorrer hacia adelante y hacia atrás.

¿Qué patrón descubrí al implementar? ¿Hay una "receta" para las operaciones de lista doble?

Se puede decir que se ocupaban de conectar entre si varias partes del código, ya sean las diferentes funciones.

Si tuviera que enseñar "listas dobles" a alguien, ¿por dónde empezaría y qué diagrama dibujaría primero?

Empezamos por el nodo y tenemos que definir qué valor tendrán dentro del nodo como la cabeza y la cola del mismo.

Fase 4: VALIDA

3 El Auditor de Listas Dobles

Documento con las 3 implementaciones erróneas, tu análisis de cada error (dónde está, por qué es incorrecto, qué consecuencia tendría en ejecución), y la versión corregida de cada método.

1) insertarFinal()

- Implementación Errónea:

```
void insertarFinal(const std::string& valor) {  
    NodoDoble* nuevo = new NodoDoble(valor);  
  
    if (cabeza == nullptr) {  
        cabeza = nuevo;  
        cola = nuevo;  
    } else {  
        // Actualizamos la cola primero para "simplificar"  
        cola = nuevo;  
  
        // Enlazamos el nuevo con el final  
        nuevo->prev = cola;  
    }  
}
```

```

cola->next = nuevo;

// Aseguramos que sea el último
cola->next = nullptr;
}

tamaño++;
}

```

- **Análisis del Error:**
 1. El problema esté en el else, se hace uso de cola=nuevo antes de usar la cola para enlazar
 2. Es incorrecto porque se pierde la referencia a la cola vieja.
- **Consecuencia de la ejecución:**
La estructura queda inconsistente, también puede que el recorrido hacia delante o atrás se quede atascado o se desconecte.
- **Versión Corregida:**

```

void insertarFinal(const std::string& valor) {
    NodoDoble* nuevo = new NodoDoble(valor);

    if (cabeza == nullptr) {
        cabeza = nuevo;
        cola = nuevo;
    } else {
        // Guardar la cola vieja (implícitamente: cola antes de cambiarla)
        nuevo->prev = cola; // 1) nuevo mira al último viejo
        cola->next = nuevo; // 2) el último viejo mira al nuevo
        cola = nuevo;      // 3) actualizar cola al nuevo
    }

    // Nuevo último: next nulo (si tu constructor no lo deja ya en nullptr,
    aquí lo aseguras)
    cola->next = nullptr;

    tamaño++;
}

```

2) eliminarNodo()

- **Implementación Errónea:**

```

void eliminarNodo(NodoDoble* nodo) {
    if (nodo == nullptr) return;

    NodoDoble* a = nodo->prev;
    NodoDoble* b = nodo->next;

```

```

if (a != nullptr) a->next = b;
if (b != nullptr) b->prev = a;

```

```

if (nodo == cabeza) {
    cabeza = b;
}

```

```

if (nodo == cola) {
    cola = a;
}

```

```

delete nodo;
tamaño--;
}

```

- **Análisis del Error:**

1. El código no valida que nodo pertenezca a la lista antes de reconectar y borrar.
2. El código asume que el nodo en ejecución dentro de su estructura es vecino valido de la misma lista, lo que se interpreta como que cualquier valor otorgado sea de este nodo.

- **Consecuencia de la ejecución:**

Puede dejar de depurar, hacer un ciclo infinito o perder segmentos

- **Versión Corregida:**

```

void eliminarNodo(NodoDoble* nodo) {
    if (nodo == nullptr) return;

```

```

    // Validar pertenencia: recorremos la lista para confirmar que el
    puntero está dentro.

```

```

    // (Costo O(n), pero evita corrupción si alguien pasa un nodo ajeno.)

```

```

    bool pertenece = false;

```

```

    for (NodoDoble* it = cabeza; it != nullptr; it = it->next) {

```

```

        if (it == nodo) { pertenece = true; break; }

```

```

    }

```

```

    if (!pertenece) return;

```

```

    NodoDoble* a = nodo->prev;

```

```

    NodoDoble* b = nodo->next;

```

```

    if (a != nullptr) a->next = b;

```

```

    else cabeza = b;           // si no hay anterior, era cabeza

```

```

    if (b != nullptr) b->prev = a;

```

```

    else cola = a;           // si no hay siguiente, era cola

```

```

        delete nodo;
        tamaño--;
    }

```

3) Método rompe simetría bidireccional

- Implementación Errónea:

```

void insertarDespuesDe(NodoDoble* pos, const std::string& valor) {
    if (pos == nullptr) return;

```

```

    NodoDoble* nuevo = new NodoDoble(valor);

```

```

    // Enganche hacia adelante
    nuevo->next = pos->next;
    pos->next = nuevo;

```

```

    // Enganche hacia atrás
    nuevo->prev = pos;

```

```

    if (pos == cola) {
        cola = nuevo;
    } else {
        // BUG: aquí pos->next ya es "nuevo", no el viejo siguiente
        pos->next->prev = pos;
    }

```

```

    tamaño++;
}

```

- Análisis del Error:

El problema está en la línea `pos->next->prev = pos;` dentro del `else`. es incorrecto porque después de `pos->next = nuevo`, `pos->next` ya no es el “viejo siguiente”, sino nuevo.

Entonces esa línea no arregla al nodo que estaba después; solo re-asigna `nuevo->prev = pos`.

- Consecuencia de la ejecución:

Hacia adelante no parece tener problemas, pero una vez regresado se pierde el valor nuevo o se lo brinca por decirlo de otra forma

- Versión Corregida:

```

void insertarDespuesDe(NodoDoble* pos, const std::string& valor) {
    if (pos == nullptr) return;

```

```

    NodoDoble* nuevo = new NodoDoble(valor);
    NodoDoble* viejoSiguiente = pos->next;

```

```

    // Conectar pos <-> nuevo

```

```

nuevo->prev = pos;
nuevo->next = viejoSiguiente;
pos->next = nuevo;

// Si había nodo después, conectarlo de vuelta hacia nuevo
if (viejoSiguiente != nullptr) {
    viejoSiguiente->prev = nuevo;
} else {
    // Si pos era la cola (no había siguiente), actualizar cola
    cola = nuevo;
}

tamaño++;
}

```

Checkpoint Metacognitivo - Fase VALIDA

¿Implementé la función verificarIntegridad() y la ejecuté después de cada operación?

¿Encontró algún error?

No, no la integré a estas operaciones, y por lo tanto no encontré errores así.

¿Pude detectar los errores en el código de la IA sin ayuda? ¿Qué estrategia usé?

No, estuve tratando muchas veces con los errores, el ultimo fue más notorio y tuve que pedir que me hiciera más preguntas para poder encontrar los bug

¿Escribí pruebas yo mismo o solo usé las que me dieron? ¿Puedo pensar en un caso de prueba que no está en la lista?

Solo use las que me dieron, me centré en encontrar los errores del bug, no en implementar otros casos.

FASE 5: PROFUNDIZA

Checkpoint Metacognitivo - Profundiza

¿Puedo identificar al menos 3 aplicaciones reales donde una lista doble es mejor que una simple?

Revisar el historial de búsqueda, y el uso del editor de texto.

¿Entiendo por qué la lista circular elimina los punteros null y qué riesgo introduce?

Si, simplifica las operaciones pero introduce un problema, que es la parada de los datos, sino se introduce un punto en el cual detenerse se expande infinitas veces sea posible

¿Puedo explicar cómo mi ListaDoble se conecta con la implementación de Pilas de la próxima semana? Pueden ser en los enlaces, solo añadiendo las conexiones y eliminando ciertos puntos para que se depure añadiendo la parada de los datos